How-To Library

Informatica

Using PowerCenter to Process Flat Files in Real Time

© Copyright Informatica LLC 2013, 2021. Informatica Corporation. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica Corporation. All other company and product names may be trade names or trademarks of their respective owners and/or copyrighted materials of such owners.

Abstract

You can use PowerCenter to process a large number of flat files daily in real time or near real time. Based on the source data, you can run a session that processes multiple flat files at scheduled intervals. Or, you can run a single real-time session that processes flat files continuously. This article presents multiple real-time or near real-time solutions that you can implement to process flat files.

Supported Versions

- PowerCenter 9.0 - 9.5.1
- B2B Data Exchange 9.0 - 9.5.1
- B2B Data Transformation 9.0 - 9.5.1

Table of Contents

Overview . . . 2
Benefits and Limitations of Flat File Processing Solutions . . . 3
PowerCenter File List . . . 4
Configuring the Session to Use a File List Generated by a Command . . . 5
B2B Data Exchange with Delayed Event Processing . . . 6
Step 1. Configure the PowerCenter Session to Use a File List . . . 7
Step 2. Create the Associated Workflow in B2B Data Exchange . . . 8
Step 3. Define Delayed Event Processing Conditions for B2B Data Exchange . . . 8
Real-time Processing . . . 10
Step 1. Generate the Source Message Queue . . . 11
Step 2. Add a JMS Source Definition to the Mapping . . . 11
Step 3. Add a Java Transformation to the Mapping . . . 12
Step 4. Create PowerExchange for JMS Connection Objects . . . 15
Step 5. Configure the Session for Real-time Processing . . . 15
B2B Data Exchange with Real-time Processing . . . 17
Step 1. Add a JMS Source Definition to the PowerCenter Mapping . . . 18
Step 2. Add an Unstructured Data Transformation to the PowerCenter Mapping . . . 19
Step 3. Create PowerExchange for JMS Connection Objects . . . 20
Step 4. Configure the PowerCenter Session for Real-time Processing . . . 21
Step 5. Export the PowerCenter Workflow . . . 22
Step 6. Create the Associated Workflow in B2B Data Exchange . . . 22

Overview

By default, a PowerCenter session reads and writes bulk data at scheduled intervals. If you process flat file data based on a time schedule, use sessions that process multiple flat files in bulk. When you configure a PowerCenter session for real-time processing, the session reads, processes, and writes data to targets continuously. If you process flat file data based on data arrival, use real-time sessions.

You can use a session that is not configured for real-time processing to read a single flat file when it arrives. However, session processing based on flat file arrival can run into the following scalability issues:

- If a workflow is trigged with each arrival of a flat file and hundreds of files arrive every minute, you might encounter a high number of concurrent workflows that can cause performance issues.
- If a single session processes one file at a time, and you need to process thousands of flat files daily, the time that it takes to reestablish the connection for each session might cause performance issues.

To solve the scalability issues, consider the following solutions to process flat files in real time or near real time:

- Run sessions that process multiple files at regular intervals.

  Use a PowerCenter file list or use B2B Data Exchange with delayed event processing.
- Run a single real-time session that reads, processes, and writes flat file data to targets continuously. Real-time sessions require messages or message queues as the real-time source. Real-time sessions must read flat file sources midstream in the pipeline.

  Use real-time processing or use B2B Data Exchange with real-time processing.

Benefits and Limitations of Flat File Processing Solutions

You can use multiple solutions to process flat files in real time or near real time. Before you choose a solution, consider your licensing options and the benefits and limitations of each solution.

PowerCenter File List

When you use a PowerCenter file list, you can run a session that processes multiple files listed in a file list.

Benefits

- Uses the PowerCenter flat file reader so that you can use all flat file reader functionality such as partitioning. If the flat file sources are large in size, you can partition the file source to increase session performance.

Limitations

- File sources must have the same format.
- Creates one session log for the entire file list, not one log for each file.
- A failure caused by one file in the file list stops the processing of all remaining files in the list.
- Processes the flat file source after a small time delay, based on how you schedule the workflow.

B2B Data Exchange with Delayed Event Processing

When you use B2B Data Exchange with delayed event processing, you can configure B2B Data Exchange to wait for a configurable number of files to arrive in a directory. B2B Data Exchange creates a file list that contains the name of each arriving file, and then starts a PowerCenter workflow to process all files listed in the file list.

Benefits

- Uses the PowerCenter flat file reader so that you can use all flat file reader functionality such as partitioning. If the flat file sources are large in size, you can partition the file source to increase session performance.

Limitations

- Creates one session log for the entire file list, not one log for each file.
- A failure caused by one file in the file list stops the processing of all remaining files in the list.
- Processes the flat file source after a small time delay, based on the delayed event processing conditions that you configure.

Real-time Processing

When you use real-time processing, you can run real-time PowerCenter sessions that read, process, and write data to targets continuously. Real-time sessions require messages or message queues as the real-time source. Real-time sessions must read flat file sources midstream in the pipeline.

Benefits

- Processes the flat file source as soon as the file arrives.
- Continues processing all files after a failure caused by one file.

Limitations

- Requires you to develop scripts to generate the source message queue.
- Creates one session log for the real-time session, not one log for each file source.
- Cannot use the PowerCenter flat file reader to partition the file source. Instead, this solution uses a Java transformation that uses a single thread to read each file in the pipeline.

B2B Data Exchange with Real-time Processing

When you use B2B Data Exchange with real-time processing, you can run PowerCenter real-time sessions that read, process, and write data to targets continuously. B2B Data Exchange uses a JMS broker to place file names in a message queue that PowerCenter uses as the real-time source. Real-time sessions must read flat file sources midstream in the pipeline.

Benefits

- B2B Data Exchange creates the message source. B2B Data Exchange watches for the file arrival and places the file name in a JMS message queue.
- Processes the flat file source as soon as the file arrives.
- Continues processing all files after a failure caused by one file.
- Provides additional logging within B2B Data Exchange.

Limitations

- Creates one session log for the PowerCenter real-time session, not one log for each file.
- Cannot use the PowerCenter flat file reader to partition the file source. Instead, this solution uses an Unstructured Data transformation available with B2B Data Transformation. The Unstructured Data transformation reads each file in the pipeline. When the sources are structured flat files that are large in size, using the PowerCenter flat file reader provides better performance than using the Unstructured Data transformation.

PowerCenter File List

With a PowerCenter file list, you can configure a session to process multiple source files for one source instance in the mapping. Use a PowerCenter file list when source files are of the same format, share the same file properties as configured in the source definition, and arrive at the same time.

A file list contains the names and directories of each source file that the PowerCenter Integration Service must read. To process flat files as they arrive, configure a command to dynamically generate the file list when the session starts. The flat file reader locates and reads the first file in the list generated by the command. After the flat file reader reads the first file, it locates and reads the next file in the list.

Use the following rules and guidelines to use the output of a command as a file list:

- Each source file must use the user-defined code page configured in the source definition.

- Each source file must share the same file properties as configured in the source definition.
- The file list must have one file name or one path and file name on a line.
- Each path in the file list must be local to the PowerCenter Integration Service node.

For more information about using a PowerCenter file list, see the *Informatica PowerCenter Workflow Basics Guide*.

PowerCenter File List Example

HypoStores Corporation uses PowerCenter to process thousands of flat files daily. The files have the same format and are large in size. HypoStores Corporation has configured partitions for the file source to increase session performance when reading the large files. However, a single session runs for each file, which causes a high session initialization time and performance issues. The files must be processed within a few minutes of their arrival.

Instead of running one session for each file, run sessions at scheduled intervals to process multiple files listed in a file list. A file list is dynamically generated every few minutes. The dynamic file list reduces the overhead of one session for each file and presents a near real-time solution. Because PowerCenter uses the flat file reader to read the files in the list, HypoStores Corporation can continue to use partitions for the file source.

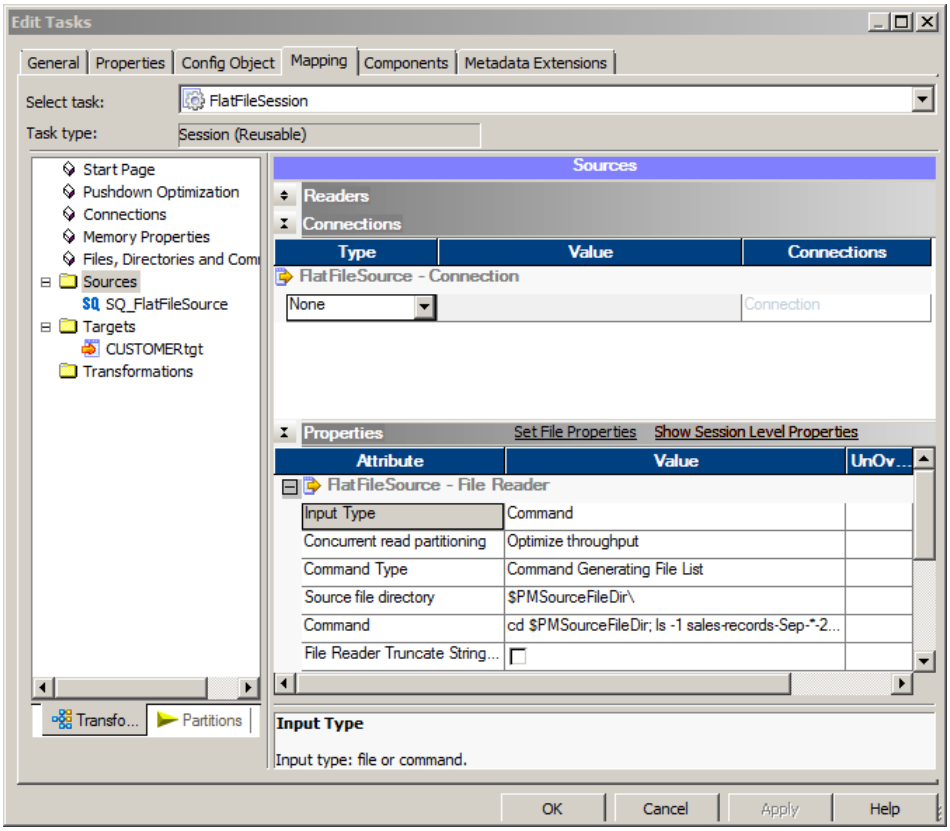
Configuring the Session to Use a File List Generated by a Command

Configure the session to use a file list that is generated by a command.

This example uses a command configured in the session properties. You can also use a command that runs outside of the session to generate a file list. For example, you can use a Command task before the session or you can use an external shell script. Then in the session properties, enter the name of the generated file list for the source file name.

1. In the Workflow Manager, open the session properties.
2. In the **Mapping** tab, click the **Sources** node.
3. In the **Properties** section, select **Command** for the input type.
4. Select **Command Generating File List** for the command type.
5. For the **Command** property, enter the command that generates the source file list from the directory that contains the arriving files. For UNIX, use any valid UNIX command or shell script. For Windows, use any valid DOS or batch file on Windows.

The following figure shows the completed properties for the **Sources** node:



6. Click **OK**.

B2B Data Exchange with Delayed Event Processing

With B2B Data Exchange with delayed event processing, you can configure B2B Data Exchange to wait for a configurable number of files to arrive in a directory. B2B Data Exchange creates a file list that contains the name of each arriving file, and then starts a PowerCenter workflow to process all files listed in the file list.

Use delayed event processing when B2B Data Exchange with real-time processing cannot be used for one of the following reasons:

- The sources are structured flat files that are large in size. The PowerCenter flat file reader provides better performance of these file types than the Unstructured Data transformation that reads files in the pipeline during real-time processing.
- For traceability reasons, you require one session log for each file list. With real-time processing, one session log is created for the PowerCenter real-time session.

To use delayed event processing to run a PowerCenter session that processes multiple files, complete the following steps:

1. In PowerCenter, configure a session to use a file list.
2. In B2B Data Exchange, create the associated workflow.
3. In B2B Data Exchange, configure delayed event processing conditions for the B2B Data Exchange profile associated with the PowerCenter workflow.

For more information about using B2B Data Exchange with delayed event processing, see the *Informatica B2B Data Exchange Operator Guide*.

B2B Data Exchange with Delayed Event Processing Example

Acme Gizmos, Inc. uses B2B Data Exchange to process flat files that it receives from business partners. Approximately 200 files arrive every 30 seconds. The files have the same format and are large in size. Acme Gizmos has configured partitions for the file source to increase session performance when reading the large files. However, B2B Data Exchange watches a directory for file arrival and starts a single PowerCenter workflow for each file, which causes a high number of concurrent workflows and performance issues. The files must be processed within 30 seconds of their arrival.

Instead of running one workflow for each file, run workflows that process multiple files in bulk. Configure B2B Data Exchange to use delayed event processing. B2B Data Exchange waits until 100 files arrive, creates a file list that contains each file name, and then starts a single PowerCenter workflow to process the file list. A file list generated every 10 to 15 seconds reduces the overhead of one workflow for each file and presents a near real-time solution. Because PowerCenter uses the flat file reader to read the files in the list, Acme Gizmos can continue to use partitions for the file source.

Step 1. Configure the PowerCenter Session to Use a File List

Configure a PowerCenter workflow with a session that uses a file list. With a PowerCenter file list, you can create a session to run multiple source files for one source instance in the mapping.

B2B Data Exchange creates the file list that contains the names and directories of each source file that PowerCenter must read. When B2B Data Exchange starts the PowerCenter workflow, it passes the file list to the workflow. The PowerCenter flat file reader locates and reads the first file in the list. After the flat file reader reads the first file, it locates and reads the next file in the list.

Use the following rules and guidelines to use a file list:

- Each source file must use the user-defined code page configured in the source definition.
- Each source file must share the same file properties as configured in the source definition.
- The file list must have one file name or one path and file name on a line.
- Each path in the file list must be local to the PowerCenter Integration Service node.

Configuring the Session to Use a File List

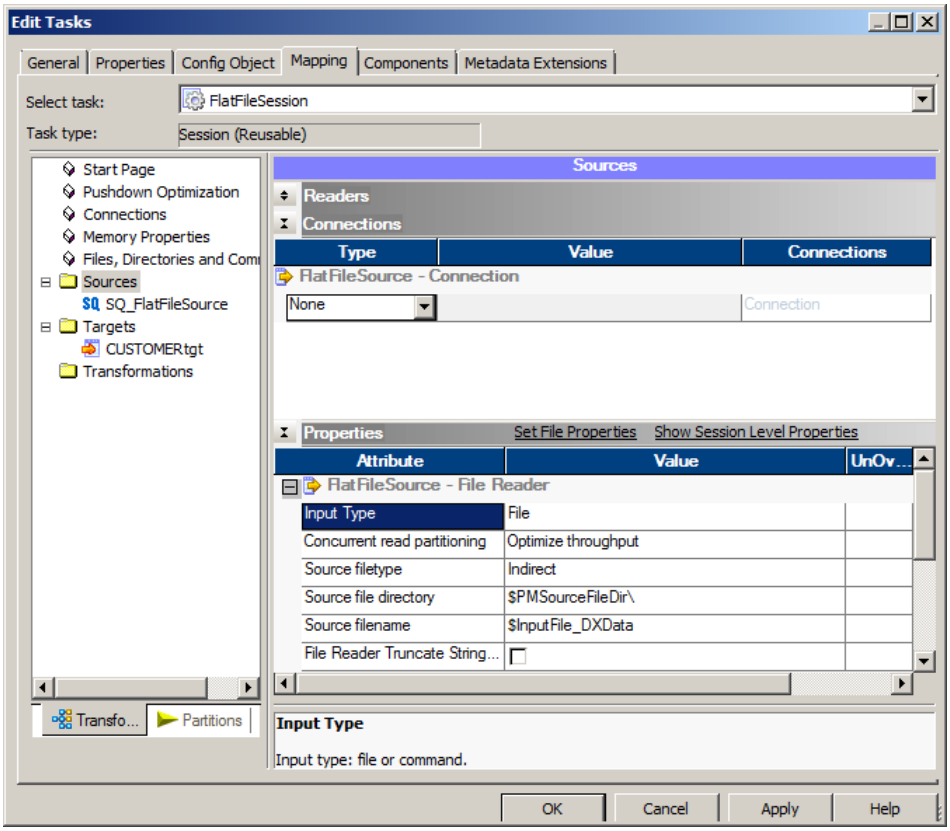
Configure the session to use the file list that B2B Data Exchange creates.

1. In the Workflow Manager, open the session properties.
2. In the **Mapping** tab, click the **Sources** node.
3. In the **Properties** section, select **File** for the input type.
4. Select **Indirect** for the source file type to indicate that the source file contains a file list.
5. Enter the following parameter for the source file name:

   ```
   $InputFile_DXData
   ```

   B2B Data Exchange passes the file list to this parameter.

The following figure shows the completed properties for the **Sources** node:



6. Click **OK**.

After you test the PowerCenter session and workflow, use the Repository Manager to export the workflow to an XML file. B2B Data Exchange requires the exported XML file to create the associated B2B Data Exchange workflow.

Step 2. Create the Associated Workflow in B2B Data Exchange

A B2B Data Exchange workflow represents a PowerCenter workflow. You must create a workflow in the B2B Data Exchange Operation Console for every PowerCenter workflow that B2B Data Exchange starts.

When you create the associated workflow in the B2B Data Exchange Operation Console, select PowerCenter batch workflow for the flow type. Then, select the exported PowerCenter workflow XML file as the workflow definition file.

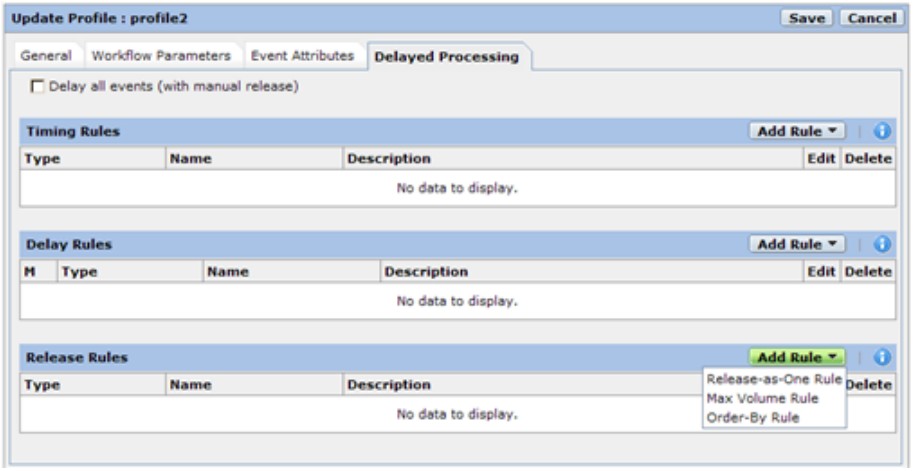
Step 3. Define Delayed Event Processing Conditions for B2B Data Exchange

In B2B Data Exchange, configure delayed event processing conditions for the B2B Data Exchange profile associated with the PowerCenter workflow. Delayed event processing uses rules to delay the events that B2B Data Exchange submits to PowerCenter.

Define a release as one rule and a maximum volume rule. The release as one rule prepares input file lists for a PowerCenter workflow. The maximum volume rule specifies that the events should be released in groups, and specifies the maximum number of events per group. For example, configure the release as one rule to prepare a file list and configure the maximum volume rule to process events after receiving 100 files. B2B Data Exchange releases the events and starts the PowerCenter workflow after receiving the configured number of files or after reaching 30 seconds, whichever occurs first.

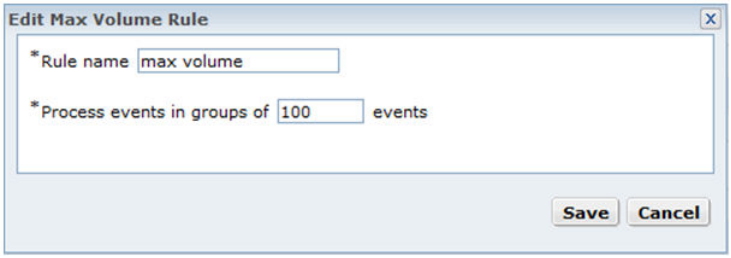
1. In the B2B Data Exchange Operation Console, click **Partner Management** > **Workflows** in the Navigator.

2. Click **Edit** for the workflow associated with the PowerCenter workflow.
3. In the **Update Workflow** page, click the **Event Attributes** tab.
4. Select the sourceDocumentType attribute key to use as an event attribute in the workflow.
5. Click **Save**.
6. Click **Partner Management** > **Profiles** in the Navigator.
7. Click **Edit** for the profile associated with the PowerCenter workflow.
8. In the **Update Profile** page, click the **Event Attributes** tab.
9. Enter `DXData` for the value of the sourceDocumentType event attribute.
10. Click the **Delayed Processing** tab.



11. Click **Release Rules** > **Add Rule** > **Max Volume Rule**.

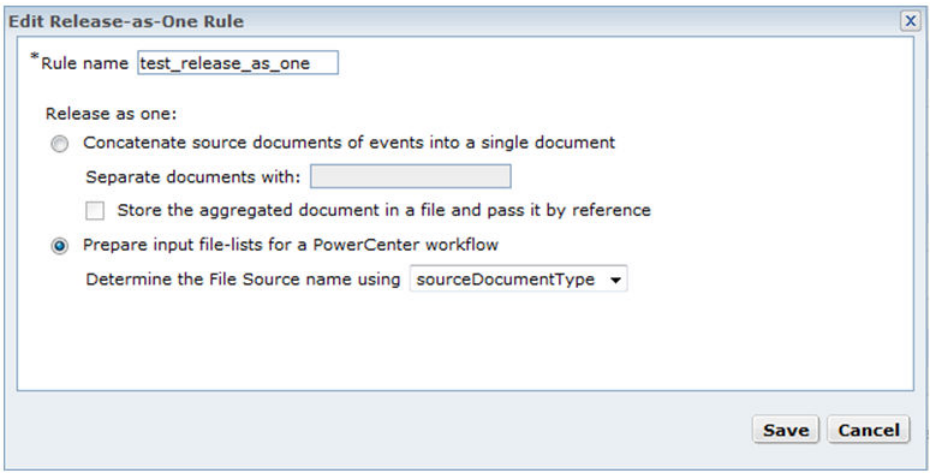
    The Max Volume Rule dialog box appears.



12. Enter a name for the rule.
13. Enter the maximum number of events per group.

    For example, enter 100.
14. Click **Save**.
15. Click **Release Rules** > **Add Rule** > **Release As One Rule**.

The Release As One Rule dialog box appears.



16. Enter a name for the rule.
17. Select **Prepare input files lists for a PowerCenter workflow**, and select the **sourceDocumentType** event attribute to determine the file source name.
18. Click **Save**.

Real-time Processing

PowerCenter real-time sessions read, process, and write data to targets continuously. Use real-time processing to read flat file sources midstream in the pipeline when the files must be processed immediately upon arrival.

You can use any of the following Informatica real-time products to process real-time source data:

- PowerExchange for JMS
- PowerExchange for TIBCO
- PowerExchange for webMethods
- PowerCenter Web Services Provider
- PowerExchange for WebSphere MQ

The examples in this article use PowerExchange for JMS.

To use real-time processing to read flat files, complete the following steps:

1. Generate the source message queue.
2. Add a JMS source definition to the mapping that reads the file path from the JMS message queue.
3. Add a Java transformation to the mapping that receives the file path as input and then reads the file.
4. Create the PowerExchange for JMS connection objects that the session uses to access the message queue.
5. Configure the real-time properties for the session.

For more information about PowerCenter real-time processing, see the *Informatica PowerCenter Advanced Workflow Guide*.

Real-time Processing Example

MegaStores Corporation uses PowerCenter to process flat files. Approximately 200 files can arrive within 30 seconds. The files arrive at different times throughout the day and are small in size. A single workflow runs for each file, which causes a high number of concurrent workflows and performance issues. The files must be processed immediately upon arrival.

Instead of running one workflow for each file, run a single workflow with a real-time session that processes files continuously. A real-time session requires real-time source data which includes messages or message queues. Develop a script to enter the file name and location of each arriving file in a JMS message queue. Add a JMS source definition to the mapping, and then add a Java transformation to read the file in the pipeline.

Step 1. Generate the Source Message Queue

Because a real-time session requires real-time source data, you must develop a script or use a messaging system to enter the file path and delimiter for each arriving file in a message queue.

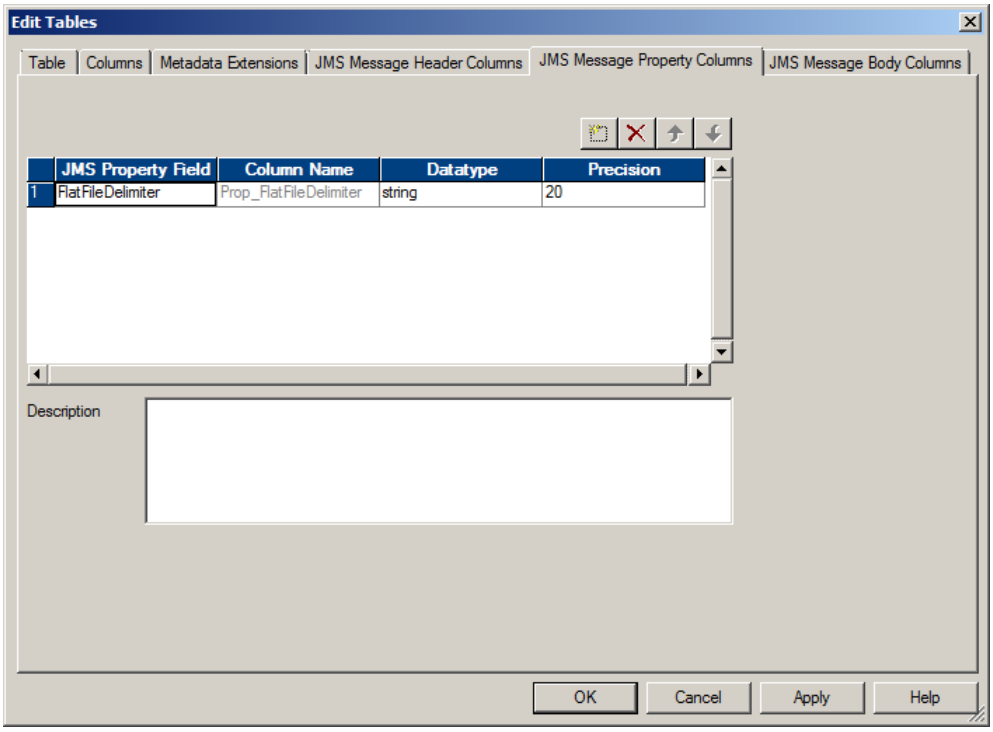
Step 2. Add a JMS Source Definition to the Mapping

Add a JMS source definition to the PowerCenter mapping so that the mapping can read the file path and delimiter from the source message queue.

1. In the Designer, click **Sources** > **Create**.
2. Enter a name for the source definition, select **JMS** for the database type, and then click **Create**.
3. In the Source Analyzer, double-click the title bar of the source definition.

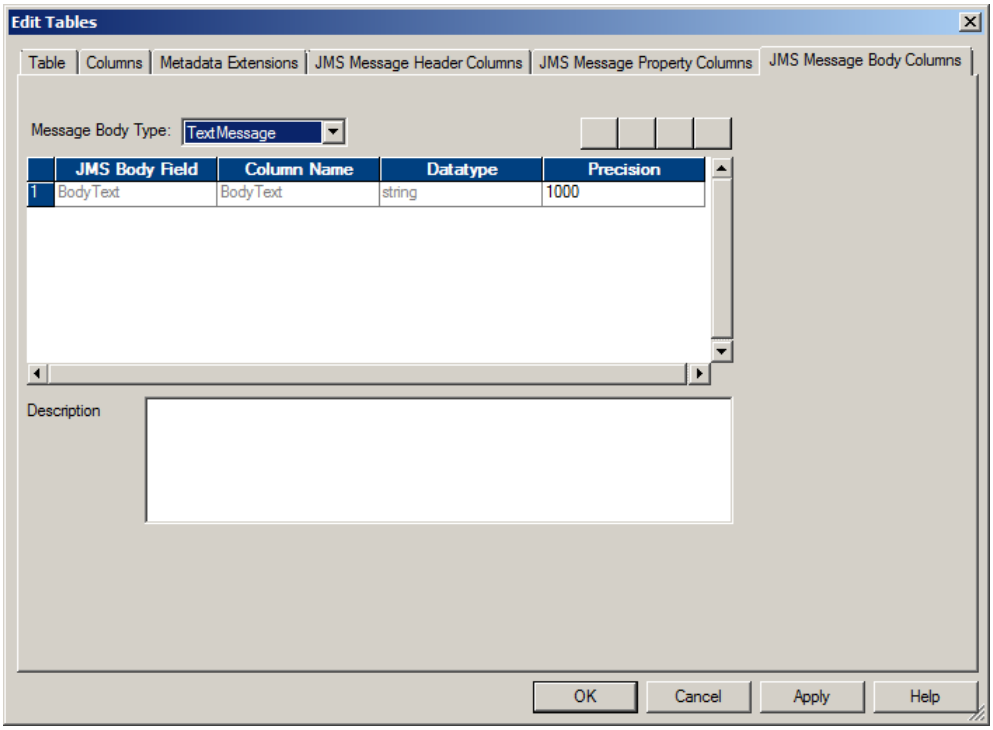
   The Edit Tables dialog box appears.
4. Click the **JMS Message Property Columns** tab.
5. Add a property column named FlatFileDelimiter.

   The FlatFileDelimiter column reads the delimiter of the flat file from the message queue.



6. Click the **JMS Message Body Columns** tab.
7. Select **Text Message** for the message body type.

The Designer adds a BodyText column to the source definition. The BodyText column reads the full file path from the message queue.



8. Click **OK**.

Step 3. Add a Java Transformation to the Mapping

Because the source message queue contains the file path and delimiter, add a Java transformation to the mapping that receives the file path and delimiter as input and then reads the file.

You can develop your own Java transformation, or you can use the example Java transformation described in this article. This example Java transformation takes the file path and delimiter of the flat file as input and then locates and reads the flat file. Each output port in the transformation represents one field in the file. This example uses third-party Java packages available from Super CSV.

This example Java transformation has the following limitations:

- All of the output ports must have a String datatype. Use an Expression transformation after the Java transformation for any datatype conversion.
- You must correctly set the port size for any field that contains data that is not a string datatype.
- In a real-time session, you must connect all of the output ports to the next transformation.
- You cannot partition the flat file source to perform parallel reads of different sections of the flat file.

By default, the Java SDK uses a maximum of 64 MB of memory during a session. If the real-time session with the Java transformation fails due to a lack of memory, you might need to increase the default value. Use the Administrator tool to modify the Java SDK Maximum Memory property for the PowerCenter Integration Service process.

Configuring the Java Transformation

Configure the Java transformation to receive the file path and delimiter as input and then read the file.

You can import the Java transformation from the following location:
https://docs.informatica.com/content/dam/source/resources/zipfiles/0441_ReadFF.zip .

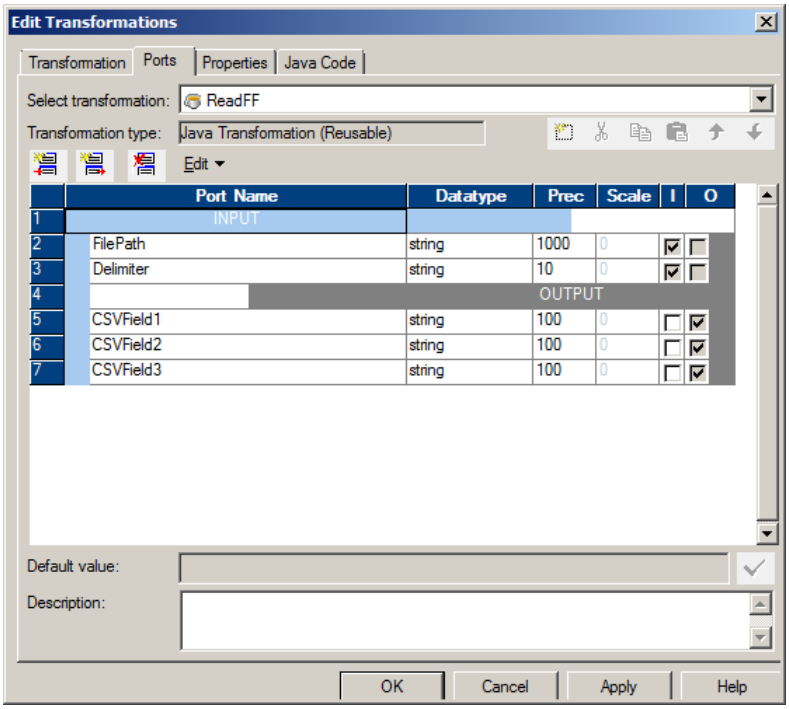
1. Download `super-csv-distribution-2.0.0-bin.zip` from the following location:
   http://sourceforge.net/projects/supercsv/.

   The Super CSV materials at the identified URL are open source materials and are being referenced as example material. Informatica is not endorsing these materials and is not responsible for the performance of or the risks posed by such materials.
2. Extract the ZIP file and then find the following JAR files in the extracted `super-csv` folder:
   - super-csv-2.0.0.jar
   - super-csv-2.0.0-javadoc.jar
   - super-csv-2.0.0-sources.jar
3. Copy the JAR files to `<Informatica Installation Directory>\server\bin\javalib`.
4. In the Designer, add a Java transformation to the mapping as an active transformation.
5. Open the Java transformation.
6. On the **Ports** tab, create the following input ports:

Port Name	Datatype	Precision
FilePath	string	1000
Delimiter	string	10

7. Create a string output port for each field in the flat file source.

The following figure shows the completed **Ports** tab for a flat file that contains three fields:



8. On the **Properties** tab, set **Transformation Scope** to **Transaction**.
9. On the **Java Code** tab, click **Settings**.
10. In the **Settings** dialog box, click **Browse** under **Add Classpath** to select the Super CSV jar files that you downloaded and copied to `<Informatica Installation Directory>\server\bin\javalib`.
11. On the **Import Packages** code entry tab, enter the following code to import the required Java and third-party packages:

```
import java.io.FileReader;
import java.util.List;

import org.supercsv.cellprocessor.Optional;
import org.supercsv.cellprocessor.ParseBool;
import org.supercsv.cellprocessor.ParseDate;
import org.supercsv.cellprocessor.ParseInt;
import org.supercsv.cellprocessor.constraint.*;
import org.supercsv.cellprocessor.ift.CellProcessor;
import org.supercsv.io.CsvListReader;
import org.supercsv.io.ICsvListReader;
import org.supercsv.prefs.CsvPreference;
```

12. On the **On Input Row** code entry tab, enter the following Java code:

```
ICsvListReader listReader = null;
   try{

      final CsvPreference CUSTOM_DELIMITED = new
CsvPreference.Builder('"',Delimiter.charAt(0), "\n").build();
      listReader = new CsvListReader(new FileReader(FilePath), CUSTOM_DELIMITED);

      //listReader.getHeader(false); // skip the header (can't be used with
CsvListReader)

      List<String> customerList;
      int numCols=grp.getOutputFieldList().size();
```

```
        while( (customerList = listReader.read()) != null ) {

           for(int i=1;i<=numCols;i++){


              if(i<=listReader.length()&&listReader.get(i)!=null)
                        outputBuf.setString(outRowNum, i-1,
listReader.get(i));
                        else
              outputBuf.setNull(outRowNum, i-1);

            }
        incrementOutputRowNumber();
           flushBufWhenFull();
        clearNullColSet();

        }

     }catch(Exception e){
     failSession("Could not read or open the specified file. Or, port could not hold the
data. Check the size of the port or the specified delimiter.");
     }
```

13. Click **Compile** to compile the Java code for the transformation.
14. Click **OK**.
15. Link the following ports from the JMS Application Source Qualifier transformation to the Java transformation:

JMS Application Source Qualifier Transformation Output Port	Java Transformation Input Port
BodyText	FilePath
FlatFileDelimiter	Delimiter

Step 4. Create PowerExchange for JMS Connection Objects

Create the application connection objects required to read from the real-time source.

In the Workflow Manager, create the application connection objects that the session requires to read source file paths from the message queue. To use PowerExchange for JMS, you must create both of the following connections:

- JNDI application connection that specifies the JNDI server that you need to access.
- JMS application connection that specifies the JMS provider that you need to access.

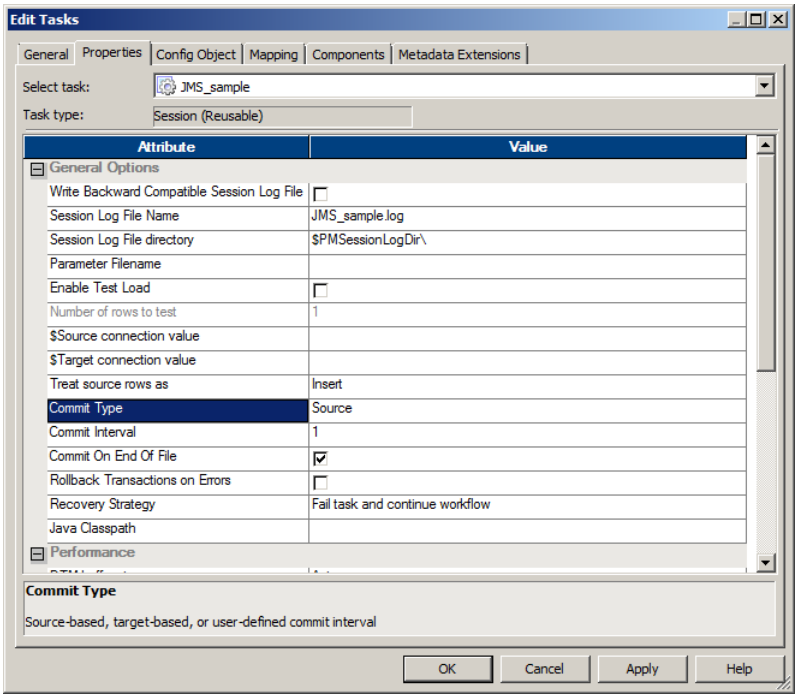
Step 5. Configure the Session for Real-time Processing

The real-time session properties control how the PowerCenter Integration Service commits data to the target and how often the PowerCenter Integration Service flushes data from the source.

1. In the Workflow Manager, open the session properties.
2. Click the **Properties** tab.
3. In the **General Options** section, select **Source** for the commit type.

   With a source-based commit, the PowerCenter Integration Service commits data based on the commit interval and the flush latency interval.
4. Enter 1 for the commit interval.

The following figure shows the completed **Properties** tab:



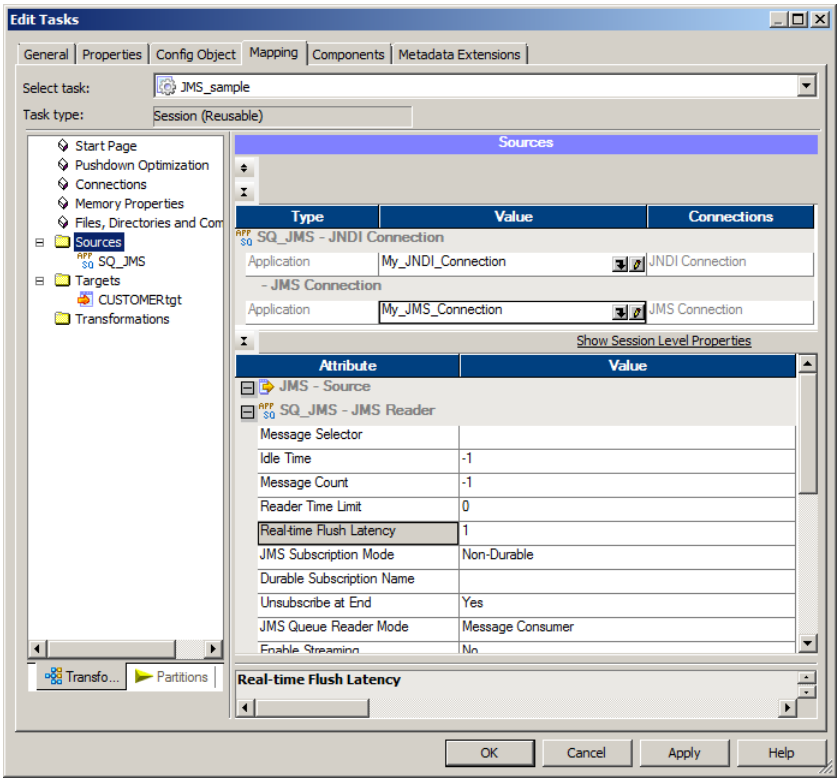
5. Click the **Mapping** tab.
6. Click the **Sources** node.
7. In the **Connections** section, select the JNDI application connection object and the JMS application connection object that you created.
8. In the **Properties** section, set the real-time flush latency to 1 or more seconds.

   Default is 0, indicating that the flush latency is disabled and the session does not run in real time.

9. Optionally, you can edit the values for the Idle Time, Message Count, and Reader Time Limit terminating conditions.

   The terminating conditions determine when the PowerCenter Integration Service stops reading from a source and ends the session. By default, the PowerCenter Integration Service reads from the source for an infinite period of time.

   The following figure shows the completed properties for the **Sources** node in the **Mapping** tab:



For more information about configuring JMS sessions and workflows, see the *Informatica PowerExchange for JMS User Guide*.

B2B Data Exchange with Real-time Processing

B2B Data Exchange with real-time processing uses a JMS broker to send files to PowerCenter for real-time processing. B2B Data Exchange watches a directory for a file arrival, places the file name in a JMS message queue, and then passes the message to a PowerCenter real-time session.

Use B2B Data Exchange with real-time processing to process flat file sources midstream in the pipeline when the files must be processed immediately upon arrival.

B2B Data Exchange uses JMS to send documents to PowerCenter real-time sessions. Use the PowerCenter Client to configure the PowerCenter mapping and session for real-time processing.

Complete the following steps to use B2B Data Exchange to run PowerCenter real-time sessions that process flat files:

1. Add a JMS source definition to the PowerCenter mapping that reads the file path from the JMS message queue.
2. Add an Unstructured Data transformation to the PowerCenter mapping that receives the file path as input and then reads the file.
3. Create the PowerExchange for JMS connection objects that the session uses to access the message queue.

4. Configure the real-time properties for the PowerCenter session.
5. Export the PowerCenter workflow to an XML file.
6. In B2B Data Exchange, create the associated workflow.

For more information about B2B Data Exchange with real-time processing, see the *Informatica B2B Data Exchange Developer Guide*.

B2B Data Exchange with Real-time Processing Example

Acme Stuff, Inc. uses B2B Data Exchange to process thousands of flat files daily that it receives from business partners. The files arrive at different times throughout the day and are small in size. B2B Data Exchange watches a directory for file arrival and starts a PowerCenter workflow and session for each file, which causes a high session initialization time and performance issues. The files must be processed immediately upon arrival.

Instead of running one PowerCenter session for each file, use B2B Data Exchange with real-time processing to run a real-time PowerCenter session to process files continuously. B2B Data Exchange watches for the file arrival, places the file name in a JMS message queue, and passes the file name to a PowerCenter workflow with a real-time session. PowerCenter uses an Unstructured Data transformation available with B2B Data Transformation to read the flat file sources in the pipeline.

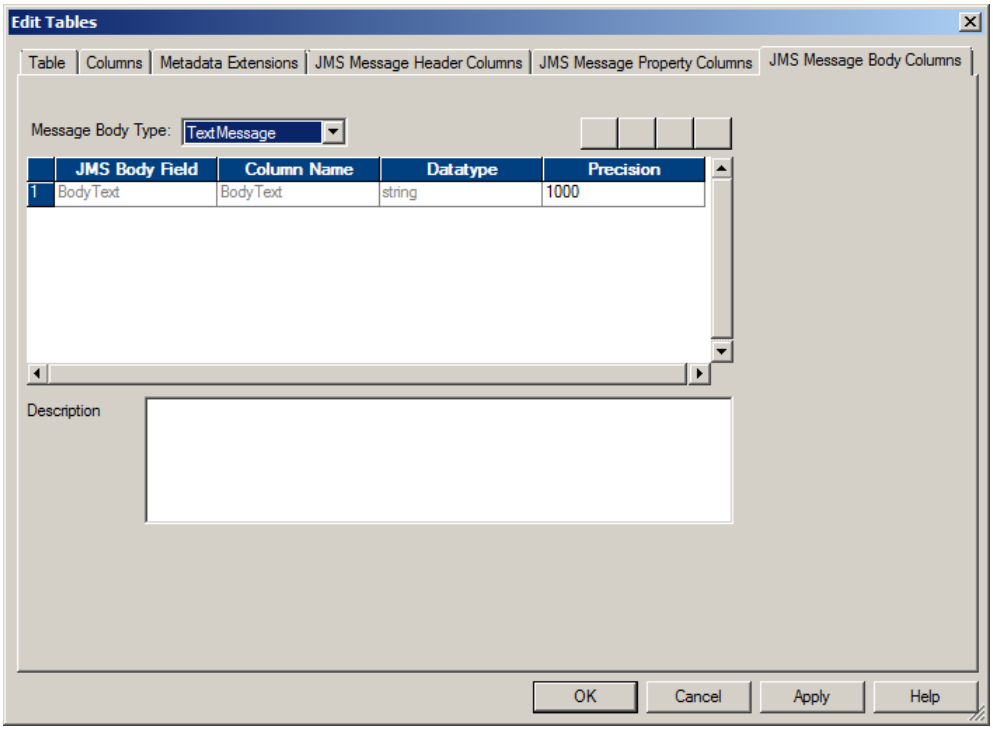
Step 1. Add a JMS Source Definition to the PowerCenter Mapping

Add a JMS source definition to the PowerCenter mapping so that the mapping can read the file path from the source message queue created by B2B Data Exchange.

1. In the PowerCenter Designer, click **Sources** > **Create**.
2. Enter a name for the source definition, select **JMS** for the database type, and then click **Create**.
3. In the Source Analyzer, double-click the title bar of the source definition.

   The Edit Tables dialog box appears.
4. Click the **JMS Message Body Columns** tab.
5. Select **Text Message** for the message body type.

The Designer adds a BodyText column to the source definition. The BodyText column reads the full file path from the message queue created by B2B Data Exchange.



6. Click **OK**.

Step 2. Add an Unstructured Data Transformation to the PowerCenter Mapping

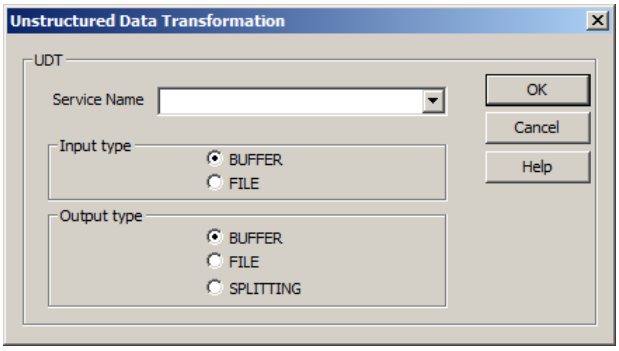
Because the source message queue contains the file path, add an Unstructured Data transformation to the PowerCenter mapping. An Unstructured Data transformation receives the source file path as input and passes the source file path to B2B Data Transformation. B2B Data Transformation reads the file and then returns the output to the Unstructured Data transformation.

The Unstructured Data transformation calls a B2B Data Transformation service from a PowerCenter session. B2B Data Transformation is an application that transforms unstructured and semi-structured file formats. You can pass data from the Unstructured Data transformation to a B2B Data Transformation service, transform the data, and return the transformed data to the pipeline.

Note: If you do not use the B2B Data Transformation application, you can use a Java transformation to read the files in the pipeline. For more information, see "Configuring the Java Transformation" on page 13.

1. In the PowerCenter Mapping Designer, click **Transformation** > **Create**.
2. Select **Unstructured Data Transformation** as the transformation type.
3. Enter a name for the transformation, and click **Create**.

The Unstructured Data Transformation dialog box appears.



4. Select the name of the Data Transformation service to run.

   The service must exist in the local Data Transformation repository.

5. Select **File** as the input type.

   The Unstructured Data transformation receives the source file path in the InputBuffer port and passes the source file path to B2B Data Transformation.

6. Select the type of output data that the Unstructured Data transformation returns to the pipeline.

7. Click **OK**.

8. Link the BodyText output port from the JMS Application Source Qualifier transformation to the InputBuffer input port in the Unstructured Data transformation.

For more information about using an Unstructured Data transformation in a PowerCenter mapping, see the *Informatica PowerCenter Transformation Guide*.

Step 3. Create PowerExchange for JMS Connection Objects

In the PowerCenter Workflow Manager, create the application connection objects that the session requires to read source file names from the JMS message queue. A JMS source requires both a JNDI application connection and a JMS application connection.

The JNDI application connection specifies the B2B Data Exchange JMS server.

The following table describes the properties of the JNDI application connection object that you must configure:

Property	Description
JNDI Context Factory	Name of the context factory specified for the B2B Data Exchange JMS provider. Enter the following value: `com.informatica.b2b.dx.jndi.DXContextFactory`
JNDI Provider URL	URL for the JNDI provider in B2B Data Exchange. The host name and port number must match the host name and port number in the jndiProviderURL attribute of the JMS endpoints in the B2B Data Exchange configuration file. For a single node installation, the JNDI provider URL is `failover:tcp://localhost:18616` by default. For an ActiveMq cluster, you can provide multiple hosts. For more information about configuring a B2B Data Exchange cluster, see the *Informatica B2B Data Exchange High Availability Guide*.

The JMS application connection specifies the input queue of the JMS source in the Data Exchange workflow. The input queue configuration must match the workflow name in B2B Data Exchange that represents the PowerCenter workflow.

The following table describes the properties of the JMS application connection object that you must configure:

Property	Description
JMS Destination Type	Type of JMS destination for the Data Exchange messages. Enter QUEUE.
JMS Connection Factory Name	Name of the connection factory in the JMS provider. Enter the following value: `connectionfactory.local`
JMS Destination	Name of the destination. The destination name must have the following format: `queue.<DXWorkflowName>` `DXWorkflowName` is the name of the workflow in B2B Data Exchange that represents the PowerCenter workflow.

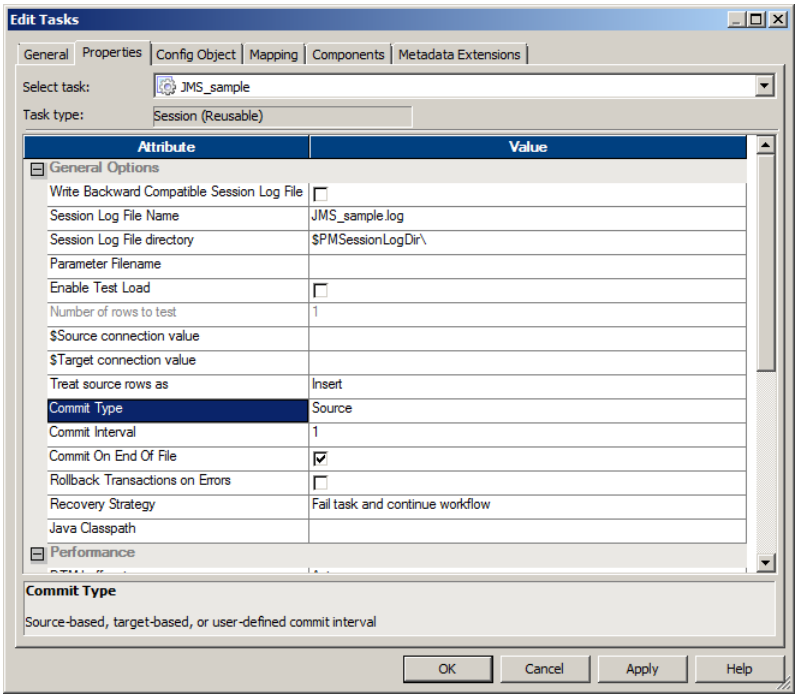
Step 4. Configure the PowerCenter Session for Real-time Processing

Configure the real-time properties for the PowerCenter session. The real-time session properties control how the PowerCenter Integration Service commits data to the target and how often the PowerCenter Integration Service flushes data from the source.

1. In the PowerCenter Workflow Manager, open the session properties.
2. Click the **Properties** tab.
3. In the **General Options** section, select **Source** for the commit type.

   With a source-based commit, the PowerCenter Integration Service commits data based on the commit interval and the flush latency interval.
4. Enter 1 for the commit interval.

   The following figure shows the completed **Properties** tab:



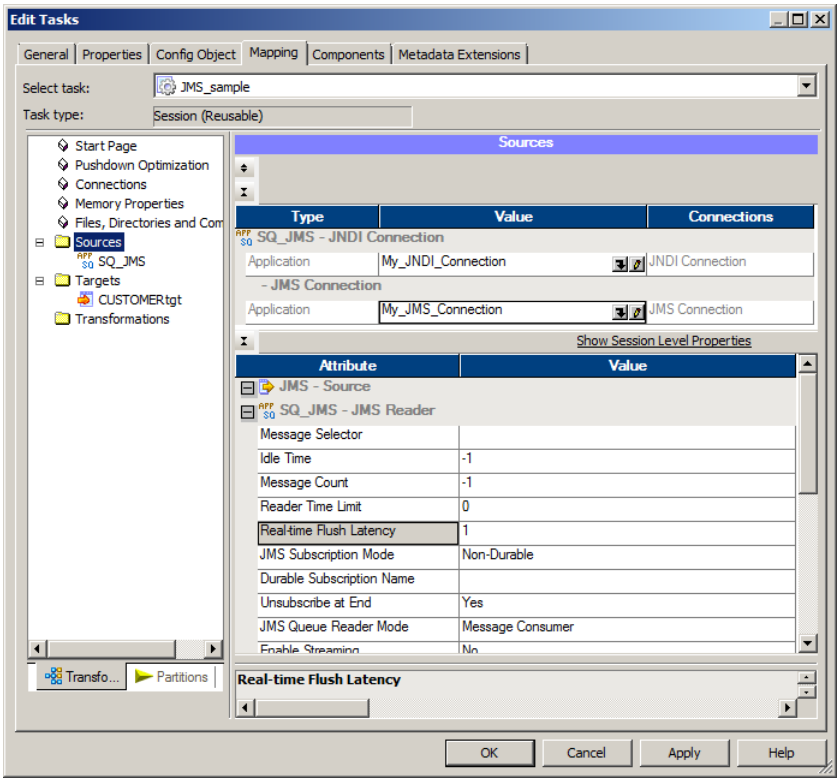
5. Click the **Mapping** tab.

6. Click the **Sources** node.
7. In the **Connections** section, select the JNDI application connection object and the JMS application connection object that you created.
8. In the **Properties** section, set the real-time flush latency to 1.

   Default is 0, indicating that the flush latency is disabled and the session does not run in real time.
9. Select **Message Consumer** for the JMS queue reader mode.
10. Optionally, you can edit the values for the Idle Time, Message Count, and Reader Time Limit terminating conditions.

    The terminating conditions determine when the PowerCenter Integration Service stops reading from a source and ends the session. By default, the PowerCenter Integration Service reads from the source for an infinite period of time.

    The following figure shows the completed properties for the **Sources** node in the **Mapping** tab:



Step 5. Export the PowerCenter Workflow

After you test the PowerCenter real-time session and workflow, use the PowerCenter Repository Manager to export the workflow to an XML file. B2B Data Exchange requires the exported XML file to create the associated B2B Data Exchange workflow.

Step 6. Create the Associated Workflow in B2B Data Exchange

A B2B Data Exchange workflow represents a PowerCenter workflow. You must create a workflow in the B2B Data Exchange Operation Console for every PowerCenter workflow that B2B Data Exchange starts.

When you create the associated workflow in the B2B Data Exchange Operation Console, select PowerCenter real-time workflow for the flow type. Then, select the exported PowerCenter workflow XML file as the workflow definition file.

Author

Alison Taylor

Acknowledgements

The author would like to acknowledge Somnath Bhadury, Anton Kuzmin, Kiran Mehta, Dinesh Rathi, and Vinutkumar Shetty for their contributions to this article.